before you dereference `old_head`. Once you've claimed the node as yours, you can clear your hazard pointer ❷. If you did get a node, you need to check the hazard pointers belonging to other threads to see if they reference it ❸. If so, you can't delete it just yet, so you must put it on a list to be reclaimed later ❹; otherwise, you can delete it right away ❺. Finally, you put in a call to check for any nodes for which you had to call `reclaim_later()`. If there are no longer any hazard pointers referencing those nodes, you can safely delete them ❻. Any nodes for which there are still outstanding hazard pointers will be left for the next thread that calls `pop()`.

Of course, there's still a lot of detail hidden in these new functions—`get_hazard_pointer_for_current_thread()`, `reclaim_later()`, `outstanding_hazard_pointers_for()`, and `delete_nodes_with_no_hazards()`—so let's draw back the curtain and look at how they work.

The exact scheme for allocating hazard pointer instances to threads used by `get_hazard_pointer_for_current_thread()` doesn't really matter for the program logic (although it can affect the efficiency, as you'll see later). So for now you'll go with a simple structure: a fixed-size array of pairs of thread IDs and pointers. `get_hazard_pointer_for_current_thread()` then searches through the array to find the first free slot and sets the ID entry of that slot to the ID of the current thread. When the thread exits, the slot is freed by resetting the ID entry to a default-constructed `std::thread::id()`. This is shown in the following listing.

Listing 7.7. A simple implementation of `get_hazard_pointer_for_current_thread()`